

An exploration problem, not an algorithm showcase

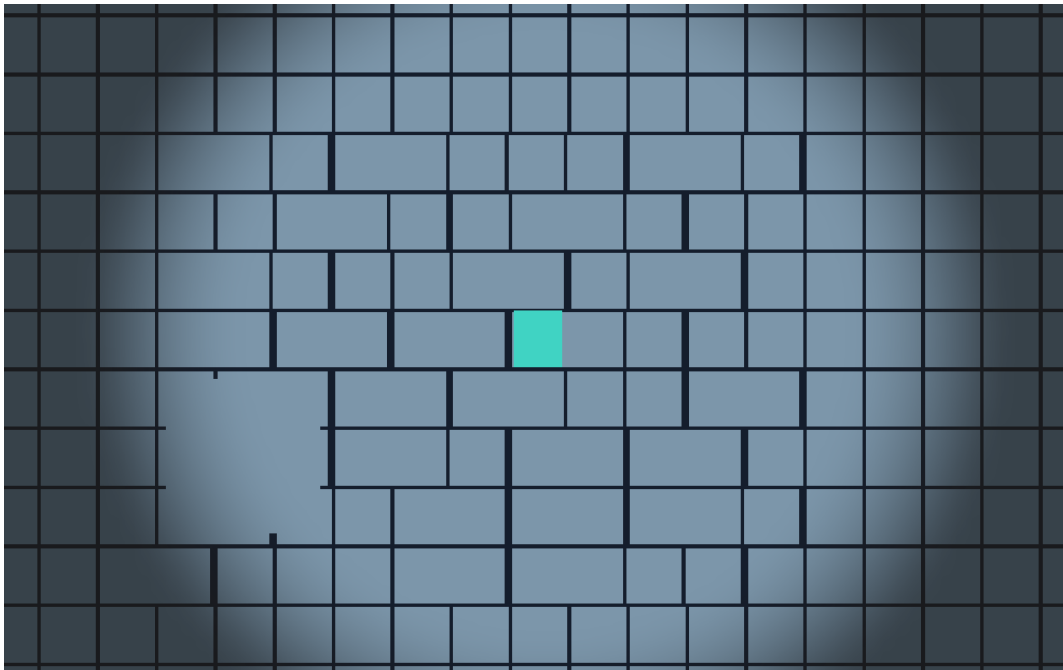
I designed and implemented systems that connect procedural space to player decisions: explore to find a key, manage damage and supplies, then reach an exit. My navigation work asks a second question: what should an assistant know when the player has not discovered the level?

ProjectMG is a team course project built in C# and Unity 6.3 LTS. My work is credited through the [hm2mty](#) commits linked below. I extended the team's map/model framework and original visual fog; I did not create the team's artwork or claim the original fog renderer, exit framework, scoring, or difficulty scaling as my own.

My contributions: room roles and key/chest/monster interactions; connecting fog state to navigation and minimap entity queries; Explore/SeekExit state logic; room-level Dijkstra and tile-level A* with danger costs and bounded healing detours; Delaunay/Kruskal generation; eight-direction input, animation timing and wall-corner repairs.

This case study distinguishes historical contributions, the team's later changes, and portfolio verification added in September 2026. Code-level checks support implementation claims; they are not evidence of a formal player study.

Implementation evidence: [51baef8](#) / [3051731](#) / [784c3a6](#) / [68677d4](#)



Actual Unity camera render / original abstract debug replacement visuals

A key gives exploration a reason



A reachable exit alone makes the level a route-following task. Requiring a key makes discovering another room necessary before the exit has value. The intended loop is spawn, reveal unfamiliar rooms, acquire the key, and return or navigate to the exit.

I added room-role assignment and key pickup using the existing Player model and team exit system. KeyItem listens to movement events; stepping on the key calls Player.PickupKey(), removes its minimap icon, and plays a scale-out pickup animation. ExitDoor advances only when the player's coordinates match the exit and HasKey is true. The key is consumed on the next level.

Current team safeguards exclude start and exit chunks from item rooms. Later placement logic selects the farthest eligible key-room center by Manhattan distance, at least 8 tiles from spawn and 10 from the exit. MapGeneratorRunner retries up to 10 times if exit or key placement fails. These later safeguards are team refinements, not solely my original placement implementation.

The design intent is separation without arbitrary backtracking. Actual placement uses 10 x 8 chunks and distance proxies, rather than a guaranteed shortest-path distance. Delaunay rooms do not exactly match those chunks. An exit and key being present does not prove they are reachable, and the retry cap can still leave a failed layout. A seed-based reachability check belongs in the release gate.

Feedback communicates state through key disappearance, a revealed minimap marker and the gameplay HUD. Pickup feedback should be verified in the player build; code inspection alone cannot establish whether a first-time player understands that the exit is now usable.

Implementation evidence: [51baef8](#) / [ed64e23](#)

Navigation should earn its knowledge

The team's visual fog hides unexplored terrain on screen. That does not automatically constrain a planner with access to MapGrid. A route to an unseen key would resolve the discovery problem before the player has explored. I connected revealed-tile queries to target selection and path construction.

FogOfWar.IsRevealed is persistent discovery memory: a tile stays revealed after the player leaves. It is not a separate currently-visible mask. Reveal uses a radius around movement, without wall-occluded line of sight; the visual soft edge also differs from the logical reveal circle. This is discovery-aware assistance, not a simulation of human vision.

In T traversal, BFS searches reachable terrain for a revealed frontier tile next to unrevealed walkable terrain. Tile A* gates intermediate steps using IsRevealed, with an exception for its target tile. Once the key is held, the state changes from Explore to SeekExit; a hidden exit keeps exploration active.

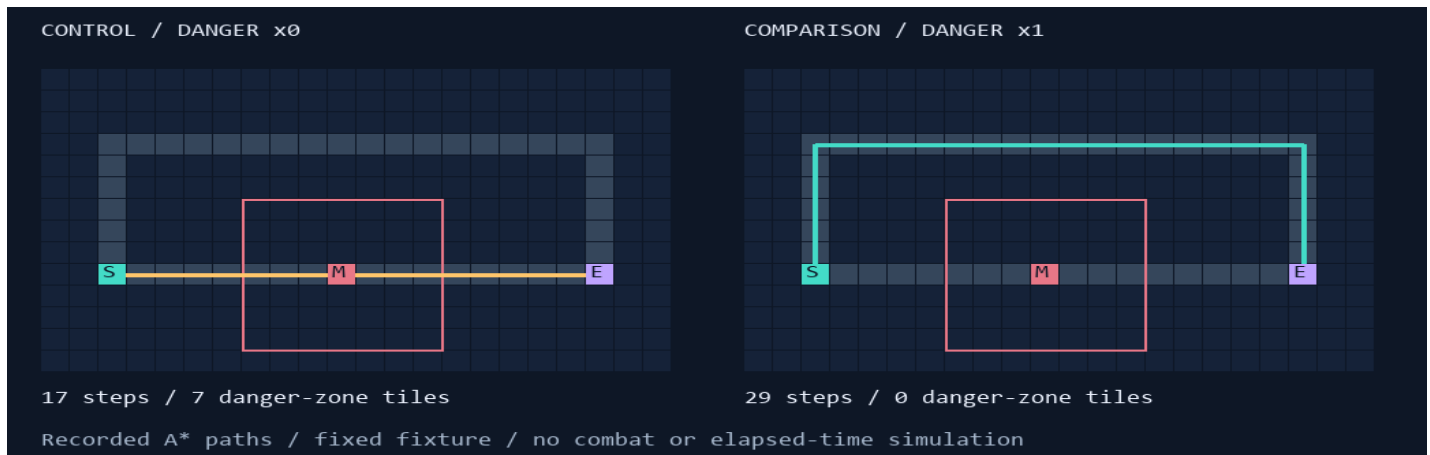
A material limitation remains: T traversal explicitly calls fog-unconstrained routing once the exit is revealed. It also detects room structure from the whole grid, and frontier selection reads tile type before checking reveal state. Both MapTraversal and GoalAI build danger fields from all active monsters. Hidden enemy positions can therefore affect a supposedly fair route.

H (Fair) and J (Efficient) GoalAI restrict normal targets and intermediate path tiles to revealed information; G is intentionally omniscient. J adds room-aware exploration and corridor safety. The modes are useful comparisons, but neither H nor J should be described as completely free of hidden information. Team GoalAI foundations and my Efficient-mode extension have separate commit credits.

Next improvement: give navigation an explicit discovered-map interface, gate danger and coin cost data at that boundary, and test that changing only hidden tiles or entities cannot change a plan. The current case documents the boundary honestly rather than claiming it is solved.

Implementation evidence: [4e0b412](#) / [3051731](#) / [68677d4](#) / [6b05c7c](#)

The shortest route can be a bad decision



A shortest route minimizes steps, not damage. I added tile danger around active monsters and a room-level surcharge to let navigation trade distance for exposure. In T traversal, DangerWeight is 18, Chebyshev radius is 3, and MonsterRoomCost is 12. These are tuning choices, not proven optimum values.

I ran the actual unmodified MapTraversal.FindPath method on one fixed 22 x 14 two-route fixture. All terrain was revealed. The control scales the computed danger field to 0; the comparison keeps it at 1, retaining the production weight of 18. Both runs use the identical start, exit, monster and geometry. The linked JSON includes every path coordinate.

The control takes 17 steps and enters 7 danger-zone tiles. The danger-weighted route takes 29 steps and enters 0 danger-zone tiles. This shows that the cost changes route choice on this fixture. It does not measure damage, completion time, remaining HP, or improvement across generated levels. It is a deterministic code experiment, not a same-seed Unity playtest.

T traversal considers revealed chests below 65% HP, only if the actual A* route is at most 18 steps. A chest heals 5 HP in the current team version. Independent checks confirmed a 6-step chest is considered at 64% HP and not at 65%. Distance gating bounds distraction, but a detour can still cost more health than it restores.

The team's GoalAI uses a separate PathPenalty of 10, HP-dependent fear, and coin attraction; my Efficient extension limits proactive chest paths and uses different critical/same-room budgets. These policies must not be conflated with T's constants. Corridor tiles inflict 10 damage per second in the current controller.

Too little danger cost collapses toward the shortest path. Too much produces long detours or apparent reluctance to cross necessary corridors. A path-step cap controls length, not expected net healing. Before claiming better survival, compare identical accepted Unity seeds, record final health and elapsed time, and separate forced corridor damage from avoidable enemy exposure.

Implementation evidence: [4e0b412](#) / [68677d4](#)

Eight directions exposed a movement contract

Observed problem: adding diagonal movement exposed inconsistent movement timing and blocked-axis behavior. Diagonal tile steps travel $\sqrt{2}$ times the distance of axis steps, and a diagonal target can be open even when adjoining wall tiles make that move clip a corner.

Reproduction: hold two movement directions on an open floor, then repeat along a wall and at a closed corner. Compare animation travel speed for horizontal and diagonal moves. An input blocked on one axis should still allow travel on the other when that destination is legal.

Cause: input direction, logical destination, and visual interpolation had different assumptions. A fixed duration per step makes diagonals visually faster. Checking only the diagonal destination does not protect wall corners. Treating a blocked combined vector as a failed move can suppress a valid axis component.

Changes: my input commit introduced eight directions and fixed axis-blocking input; the animation commit scaled move duration by actual distance. The corner fix checks horizontal, vertical and diagonal destinations. It allows a diagonal only if all three are open, otherwise falls back to an open axis. The current controller prefers horizontal fallback if both axes are open but the diagonal is blocked.

Verification: five independent portfolio regression checks execute the actual current `PlayerController.Tick`: open diagonal, closed-corner rejection, horizontal fallback, vertical fallback and horizontal priority when the diagonal is blocked. All passed. The animation-duration scaling is verified by source inspection; the harness does not measure rendered animation. These are current-code checks, not records of a historical player study. A player-build check must still confirm interpolation and camera feel.

Remaining limits: manual movement supports eight directions while auto-navigation uses four. Axis fallback has a directional bias. Animation accepts another movement near 90% completion, which can affect perceived cadence. These are concrete next observations for a playtest, not reasons to rework the art or add unrelated mechanics.

Implementation evidence: [06a5f01](#) / [c5f0f8a](#) / [2bc1c68](#)

Inspect the work and reproduce the evidence

Source: HachimiGame/ProjectMG on GitHub. Baseline reviewed:

6ed95c17226cbba26273d56ad8dab2eab1b4c662. Historical implementation claims link to individual commits rather than treating every current feature as my work.

September 2026 verification: the Windows x64 player built successfully in Unity 6000.3.8f1. Automated J, H, G and T runs all reached Level 2 on accepted Unity seed 5800. These first-level checks do not validate the whole campaign or manual input feel. The downloadable debug edition replaces original art with abstract swatches; its release status records the exact verification scope.

Controls in the current project: WASD / arrow-key movement; T toggles two-phase traversal; G toggles omniscient GoalAI; H toggles Fair GoalAI; J toggles Efficient GoalAI. R restarts after the final completion screen. Start from SampleScene. Check the included run guide for mode limitations and release verification.

Portfolio screenshots and diagrams must not imply a measured player study. The risk figure is drawn directly from recorded path coordinates in the code experiment. The generator figure uses unmodified Delaunay/Kruskal code with an explicitly substituted System.Random adapter; it is not a Unity screenshot or a reproduction of Unity's seed 5800.

The repository licenses code under MIT but reserves artwork rights. The isolated distribution preparation replaces all 278 raster assets with original abstract debug swatches, preserving sprite-sheet dimensions. It does not grant permission to redistribute the original art. Review the replacement audit and third-party notices before releasing a binary.

Verification added for this portfolio is labeled separately from the course work. No formal external playtest or clean-machine test is claimed. Video recording is intentionally outside this delivery; a 2-3 minute English recording script and caption plan are included separately.

Release criteria: a verified spawn-to-key-to-exit run, useful fog-constrained mode behavior, published download integrity, and launch on a machine without a Unity development environment. Any check not completed is retained as pending in the delivery status, rather than silently marked passed.